

TileGen to Unreal Engine: Complete Beginner Workflow

Purpose of This Guide

This guide walks you through the complete process of creating a simple tile-based map with **TileGen** and importing that map into **Unreal Engine 5.8.3**.

It is written for a beginner who may not be familiar with TileGen, Unreal Engine asset pipelines, Python editor scripts, or generated C++ map data.

By the end of this guide, you should be able to:

- run TileGen;
- generate a few simple 64×64 tile images;
- create a basic 64×64 map;
- export the map as C++ data;
- generate TileGen's Unreal import script;
- import the tile textures into Unreal;
- configure the tiles for crisp pixel-style rendering;
- add the generated map files to the Unreal C++ project;
- create a simple tile material;
- create a basic C++ tile-map renderer;
- place the renderer into the **PlayGame** level;
- add a temporary overhead camera;
- launch Unreal and see the generated TileGen map.

The purpose of this first pass is **not** to create a finished game.

The purpose is to prove that the entire pipeline works:

TileGen → Unreal Assets → Generated C++ Data → C++ Renderer → Visible Map

Before You Begin

You should have:

- Windows PC
- Unreal Engine 5.8.3 installed
- Visual Studio installed with C++ game-development support
- the TileGen repository ZIP
- enough disk space for Unreal project files and packaged builds
- an Unreal C++ project for the game

For the examples below, this guide assumes your Unreal project is named:

GrantMoney

and that your main gameplay level is:

PlayGame

Your project may use different names, but you should substitute your own names consistently.

Step 1 — Extract and Run TileGen

You were provided a ZIP containing the TileGen repository.

Extract the repository somewhere easy to work with.

Avoid putting the project inside deeply nested folders or synchronized cloud folders such as OneDrive if possible.

A simple location might be:

```
C:\GameGuru\TileGen\
```

Inside the extracted repository, locate:

```
TileGenBinaries.zip
```

Extract that ZIP as well.

Then run:

```
TileGen.exe
```

If TileGen Does Not Start

TileGen uses .NET 8.

If Windows tells you that a runtime is missing, install the:

.NET 8 Desktop Runtime x64

Then launch `TileGen.exe` again.

You do not need to compile TileGen from source just to use it.

Step 2 — Understand the Main TileGen Tabs

TileGen contains several major tools:

```
Tile Generator
Tile Blender
Map Generator
Map Colorizer
```

For this first map, you only need:

```
Tile Generator
Map Generator
```

You will also use one Unreal-related export option from the Map Colorizer area later.

For now, ignore:

- advanced Tile Blender workflows;
- complex colorization;
- decoration generation;
- collision generation;
- JSON exports;
- CSV exports;
- atlases;
- advanced procedural effects.

The goal is to prove the basic pipeline first.

Step 3 — Create TileGen Output Folders

Create a simple folder structure outside your Unreal project.

For example:

```
C:\GameGuru\TileGen\
```

Inside it create:

```
C:\GameGuru\TileGen\Tiles
C:\GameGuru\TileGen\Maps
```

Use:

```
Tiles
```

for generated PNG textures.

Use:

```
Maps
```

for generated C++ map files and the Unreal import script.

Keeping these separate makes it much easier to understand what TileGen is producing.

Step 4 — Generate a Few Simple Tiles

Open the:

Tile Generator

tab.

Set the output directory to:

```
C:\GameGuru\TileGen\Tiles
```

For your first test, create only two or three tiles.

You are testing the workflow, not making final artwork.

Example Tile 1 — Grass

Choose colors such as:

```
Color 1: Dark Green  
Color 2: Light Green  
Algorithm: Perlin
```

Then click:

Generate & Save Tile

Example Tile 2 — Dirt

Use something like:

```
Color 1: Brown
Color 2: Tan
```

Generate and save it.

Example Tile 3 — Stone

Use:

```
Color 1: Dark Gray
Color 2: Light Gray
```

Generate and save it.

TileGen creates 64×64 PNG files.

Your folder may now look similar to:

```
Tiles/
  tile_20260928_183400_001.png
  tile_20260928_183512_002.png
  tile_20260928_183610_003.png
```

Important Rule

Do not casually rename these files after generating a map.

TileGen associates map tile IDs with tile filenames.

For example:

```
Tile 1 = grass texture
Tile 2 = dirt texture
Tile 3 = stone texture
```

If the filenames change later, you can break the relationship between the generated map data and the texture lookup.

Step 5 — Open the Map Generator

Go to the:

Map Generator

tab.

Locate the setting that allows you to select your tile directory.

Choose:

```
C:\GameGuru\TileGen\Tiles
```

TileGen should load your generated PNG files into the tile palette.

If you generated three tiles, you should see approximately three selectable tile thumbnails.

If no tiles appear:

1. verify the folder path;
2. verify the PNG files are really inside that folder;
3. confirm that the files are valid PNG images;
4. restart or refresh TileGen if necessary.

Step 6 — Configure the Map Output

Within the Map Generator, find the map-output settings.

For the first test use:

```
Level: 1
```

Set the map output directory to:

```
C:\GameGuru\TileGen\Maps
```

This is where TileGen will write the generated C++ map files.

Keeping map data separate from texture files makes the export package easier to inspect and version.

Step 7 — Generate the Simplest Possible Map

For your first test, avoid complex procedural settings.

Under the map-generation options, select:

Open arena

You may also enable:

Border wall

if you want a visible boundary.

Then click:

Generate Map

TileGen will create the logical shape of the map.

At this stage, TileGen has generated where valid map cells exist.

It may not yet have assigned your grass, dirt, or stone textures to all of those cells.

Think of this as creating the map's structure before painting it.

Step 8 — Paint the Ground Layer

Make sure the active map layer is:

Ground

Select a tile from the tile palette.

For example, select your grass tile.

Choose the:

Fill

tool.

Click inside the generated playable area.

The valid connected area should fill using the selected ground tile.

You can then select another tile and use tools such as:

```
Paint
Rectangle
Line
```

to create simple variation.

For example:

```
GGGGGGGGGGGG
GGGGGGGGGGGG
GGGGDDDDGGGG
GGGGDDDDGGGG
GGGGDDDDGGGG
GGGGGGGGGGGG
GGGGGGGGGGGG
```

where:

```
G = grass
D = dirt
```

This is enough for the first Unreal test.

Do not spend time making a polished map yet.

Step 9 — Save the Map

Click:

Save Map

TileGen will generate C++ files similar to:

```
Map1_1.cpp
Map1_1.h
Map1_1_TileAssets.h
```

If you save another version, TileGen may generate names such as:

```
Map1_2.cpp
Map1_2.h
```

```
Map1_2_TileAssets.h
```

instead of overwriting the first one.

What These Files Contain

Map1_1.cpp

This contains the generated map arrays.

Conceptually:

```
Map1_1[x][y]
```

tells the game which ground tile ID exists at a specific map coordinate.

Other generated arrays may include information such as:

```
Map1_1_Valid[x][y]
```

which identifies whether a cell belongs to the playable map.

You may also see arrays for:

```
Map1_1_Deco[x][y]  
Map1_1_Collision[x][y]
```

depending on the export.

Map1_1_TileAssets.h

This file connects numeric tile IDs to Unreal asset paths.

Conceptually:

```
0 = no tile  
1 = /Game/Tiles/grass  
2 = /Game/Tiles/dirt  
3 = /Game/Tiles/stone
```

This file is the bridge between:

```
Map says "Tile 2"
```

and:

```
Unreal loads the correct texture for Tile 2
```

Step 10 — Generate the Unreal Tile Import Script

TileGen can generate a Python script that imports the PNG tile library into Unreal automatically.

The generated file is typically named:

```
import_tiles_to_unreal.py
```

In the version of TileGen used for this project, the control for generating this script may be located under the:

Map Colorizer

area.

You do not need to use the Map Colorizer itself.

Keep your normal TileGen tile directory set to:

```
C:\GameGuru\TileGen\Tiles
```

Set the relevant export/output path to:

```
C:\GameGuru\TileGen\Maps
```

Then use the:

Unreal import script

option.

Afterward, your Maps folder should contain something similar to:

```
Maps/  
  Map1_1.cpp
```

```
Map1_1.h  
Map1_1_TileAssets.h  
import_tiles_to_unreal.py
```

This Python script is used only inside the Unreal Editor to automate tile import and texture configuration.

It is not part of the final game runtime.

Step 11 — Prepare the Unreal Project

Open your Unreal C++ project.

For this guide, the project is assumed to be:

```
GrantMoney
```

Inside the Unreal Content Browser, create this simple structure if it does not already exist:

```
Content/  
  Maps/  
  Tiles/  
  Materials/  
  UI/  
  Audio/
```

Your main level should be:

```
Content/Maps/PlayGame
```

For the first import, it is helpful if:

```
Content/Tiles
```

is empty.

This makes it easy to verify exactly what the TileGen Python script imports.

Step 12 — Enable Unreal Python Editor Support

TileGen's Python script runs inside Unreal Editor.

Go to:

```
Edit → Plugins
```

Search for:

```
Python Editor Script Plugin
```

Enable it.

Also search for:

```
Editor Scripting Utilities
```

Enable that as well.

Unreal may ask you to restart the Editor.

If it does, save your work and restart.

Important Concept

Python is being used here only as an editor automation tool.

The finished game does not require Python to run.

The script performs repetitive asset-import operations during development, and the imported textures become ordinary Unreal assets afterward.

Step 13 — Run the TileGen Import Script

Before executing the script, open:

```
import_tiles_to_unreal.py
```

in a text editor.

Look for configuration values similar to:

```
SOURCE_DIR = r"C:\GameGuru\TileGen\Tiles"  
DEST_PATH = "/Game/Tiles"
```

Make sure:

```
SOURCE_DIR
```

points to the folder containing your generated PNG files.

Make sure:

```
DEST_PATH
```

points to:

```
/Game/Tiles
```

Save the script if you changed the path.

Then return to Unreal.

Use:

```
Tools → Execute Python Script
```

Select:

```
import_tiles_to_unreal.py
```

Allow the script to finish.

Then open:

```
Content/Tiles
```

in the Content Browser.

You should see the imported tile textures.

Verify the Import

Count the PNG files in:

```
C:\GameGuru\TileGen\Tiles
```

Then compare that number with the number of imported Unreal textures.

They should match.

Do not assume success just because some textures appeared.

Step 14 — Verify Texture Settings

Open several imported tile textures inside Unreal.

Check the following:

```
Dimensions: 64 × 64  
sRGB: Enabled  
Filter: Nearest  
Mip Gen Settings: NoMipmaps
```

The tiles should look crisp when zoomed in.

You should not see blurry interpolation between pixels.

Check more than one texture.

The goal of the automated import script is to make these settings consistent across the entire tile library.

If some imported tiles have different settings, investigate the script or the import result before moving forward.

Step 15 — Verify Reimport Behavior

A repeatable import pipeline is more useful than a one-time import.

Choose one generated tile PNG.

Create a backup before experimenting.

Modify or regenerate that tile while keeping the same filename.

Run:

```
import_tiles_to_unreal.py
```

again.

Check:

Content/Tiles

The existing Unreal asset with the same name should update instead of creating a duplicate.

Also verify that introducing a brand-new PNG filename creates a new Unreal asset.

Why This Matters

Your generated map refers to tile IDs that eventually resolve to specific filenames.

Keeping stable filenames allows you to update the art without changing the map's numeric data.

Avoid filenames such as:

```
grass_new.png  
grass_final.png  
grass_final2.png  
grass_really_final.png
```

unless you actually intend them to represent different tile identities.

Step 16 — Ensure Tile Assets Are Included in Packaged Builds

Because your C++ renderer may load tile assets dynamically, Unreal may not automatically detect every texture as a packaging dependency.

Go to:

Edit → Project Settings → Packaging

Find:

Additional Asset Directories to Cook

Add:

```
/Game/Tiles
```

Save the project settings.

This tells Unreal to include the tile textures when cooking and packaging the game.

Without this step, a map might work perfectly in the Editor but display missing textures in a packaged build.

Step 17 — Add the Generated Map Files to the Unreal C++ Project

Inside your Unreal project source tree, create a folder such as:

```
Source/  
  GrantMoney/  
    Maps/
```

Copy the generated files into that folder:

```
Map1_1.cpp  
Map1_1.h  
Map1_1_TileAssets.h
```

The result should look similar to:

```
GrantMoney/  
  Maps/  
    Map1_1.cpp  
    Map1_1.h  
    Map1_1_TileAssets.h
```

Rebuild the Unreal C++ project.

If Visual Studio does not show the new files automatically, regenerate the Unreal project files or reload the solution.

At this point, the TileGen map data is part of the Unreal C++ project.

You still need a renderer to make that data visible.

Step 18 — Understand What the Generated Map Data Means

Suppose the generated map contains:

```
Map1_1[5][8] = 2;
```

Conceptually this means:

```
At coordinate X = 5 and Y = 8,  
the ground tile ID is 2.
```

The generated tile-asset lookup might associate:

```
Tile ID 2
```

with:

```
/Game/Tiles/some_dirt_texture
```

The renderer therefore performs this process:

```
Read map coordinate  
    ↓  
Get tile ID  
    ↓  
Resolve tile ID to Unreal texture  
    ↓  
Create/display a tile at that world position
```

The generated array may look intimidating because it contains thousands of numbers.

In reality, it is simply a grid of tile IDs.

You can think of it like a spreadsheet:

```
X,Y → Tile Number
```

Step 19 — Create a Simple Unreal Tile Material

Before writing the renderer, create a material in:

```
Content/Materials
```

Name it:

```
M_Tile
```

Open the material.

For the simplest possible tile renderer, set:

```
Shading Model: Unlit
```

Add a:

```
Texture Sample Parameter 2D
```

Name the texture parameter:

```
TileTexture
```

Connect:

```
RGB → Emissive Color
```

Save the material.

Why Use Unlit First?

The first objective is to prove that:

```
TileGen map data
```

can successfully produce:

```
visible Unreal tiles
```

Lighting introduces another variable.

Using an unlit material lets you see the texture directly without needing to solve lighting at the same time.

You can replace or improve the material later.

Step 20 — Create the C++ TileMapRenderer

In Unreal, create a new C++ class.

Use:

```
Actor
```

as the parent class.

Name the new class:

```
TileMapRenderer
```

The first version of this renderer only needs to do the following:

```
Read the generated map
  ↓
Loop through X and Y coordinates
  ↓
Check whether a cell should exist
  ↓
Read the tile ID
  ↓
Resolve the corresponding tile asset
  ↓
Place a flat tile at the correct Unreal position
```

Conceptually:

```
for (int x = 0; x < 64; x++)
{
    for (int y = 0; y < 64; y++)
    {
```

```
    if (!Map1_1_Valid[x][y])
    {
        continue;
    }

    int TileId = Map1_1[x][y];

    if (TileId == 0)
    {
        continue;
    }

    FVector Position(
        x * 100.0f,
        y * 100.0f,
        0.0f
    );

    // Resolve TileId to a texture.
    // Add a flat tile instance at Position.
}
}
```

The exact production implementation may differ.

The important idea is:

TileGen coordinate → Unreal world position

For a simple first test, using:

100 Unreal units per tile

is convenient.

That makes a 64×64 map approximately:

6400 × 6400 Unreal units

Recommended Rendering Approach

Do not create 4,096 independent Actor objects if you can avoid it.

A better long-term implementation is to use:

Instanced Static Meshes

or:

Hierarchical Instanced Static Meshes

grouped by tile type.

However, if you are only proving the concept, start with the simplest version you can understand and verify.

Step 21 — Test in Stages, Place the Renderer, and Add a Temporary Camera

Do not attempt to debug:

- tile IDs;
- generated textures;
- map orientation;
- materials;
- camera;
- collision;

all at once.

Test the renderer in stages.

Stage A — Render Valid Cells Only

Ignore textures initially.

For every valid map cell, place a plain white square.

Verify:

- the shape matches TileGen;
- X and Y are not reversed;
- the map is not rotated incorrectly;
- the map is not transposed.

If the map shape is wrong, fix coordinate handling before adding texture logic.

Stage B — Use Tile IDs

Once the map geometry is correct, use:

```
Map1_1[x][y]
```

to read actual tile IDs.

Verify that different parts of the map use the correct IDs.

Stage C — Load the Actual Textures

Now use:

```
Map1_1_TileAssets
```

or the project-level lookup interface to resolve each tile ID to its Unreal texture.

Apply the texture through the tile material.

Verify that the rendered map visually matches the map you created in TileGen.

Place the Renderer in the PlayGame Level

Compile the C++ project.

Return to Unreal Editor.

Find your C++ renderer class under something similar to:

```
C++ Classes/  
  GrantMoney/  
    TileMapRenderer
```

Drag:

```
TileMapRenderer
```

into:

```
PlayGame
```

This is still a C++ gameplay implementation.

You are simply placing a C++ Actor instance into the Unreal level.

Add a Temporary Overhead Camera

Do not build the final camera system yet.

For the first rendering test, add a standard:

```
Camera Actor
```

to the **PlayGame** level.

A reasonable starting location might be:

```
X = 3200  
Y = 3200  
Z = 7000
```

Set the camera rotation approximately to:

```
Pitch = -90°  
Yaw   = 0°  
Roll  = 0°
```

For a tile-map-style test, you may use:

```
Projection Mode = Orthographic
```

with something like:

```
Ortho Width = 7000
```

Set:

```
Auto Activate for Player = Player 0
```

These are only starting values.

Adjust them if your generated map uses a different tile spacing or map scale.

Final Test

Press:

Play

You should now see your generated TileGen map from above.

The map should follow the same overall layout you created in TileGen.

At this point, the full basic pipeline is working:

```
TileGen
  ↓
Generate PNG tiles
  ↓
Generate 64×64 map
  ↓
Export C++ arrays
  ↓
Generate Unreal Python importer
  ↓
Import PNGs into /Game/Tiles
  ↓
Compile generated map files
  ↓
TileMapRenderer
  ↓
Resolve tile IDs to textures
  ↓
Place tiles in Unreal world
  ↓
View with overhead camera
```

What Not to Work on Yet

For this first milestone, avoid adding:

- Tile Blender workflows
- decoration layer
- collision layer
- JSON map loading
- CSV loading
- tile atlases
- complex procedural-map effects
- advanced lighting
- final camera controls
- player character

- movement
- enemies
- combat
- collectibles
- UI
- multiple finished maps

Those features can be added after the basic TileGen-to-Unreal pipeline is proven.

Beginner Milestone Checklist

You are finished with this first workflow when all of the following are true:

- [] TileGen launches successfully.
- [] At least two or three 64×64 PNG tiles were generated.
- [] TileGen loads those tiles in Map Generator.
- [] A simple open-arena map was generated.
- [] The map contains visible ground-tile variation.
- [] The map was exported to C++ files.
- [] Map1_1.cpp exists.
- [] Map1_1.h exists.
- [] Map1_1_TileAssets.h exists.
- [] import_tiles_to_unreal.py exists.
- [] Unreal Python Editor Script Plugin is enabled.
- [] Editor Scripting Utilities is enabled.
- [] The Python script imports the tile PNGs into /Game/Tiles.
- [] Imported tiles are 64×64.
- [] Imported tiles use nearest filtering.
- [] Imported tiles have mipmaps disabled.
- [] /Game/Tiles is included for cooking.
- [] Generated map files compile inside the Unreal project.
- [] The renderer can identify valid map cells.

- [] The renderer can identify tile IDs.
- [] Tile IDs resolve to the correct Unreal texture assets.
- [] TileMapRenderer is placed in PlayGame.
- [] An overhead camera can see the generated world.
- [] The rendered map matches the TileGen map layout.

Recommended Next Step

Once this works, the next development milestone should be:

```
Generated Map Data
  ↓
Reusable Map Renderer
  ↓
C++ Overhead Camera
  ↓
Camera Movement
  ↓
Zoom
  ↓
World Boundaries
  ↓
Multiple Map Validation
  ↓
Lighting
  ↓
Packaged Build
```

Only after that should you move into the next major game-development module:

```
Player Character
Movement
Animation
Camera Integration
```

The most important rule throughout this process is:

Prove one layer of the pipeline before adding the next layer.

That keeps failures understandable and makes the workflow much easier to teach, document, and reproduce.